Context-Aware Vocabulary Pruning for LLM Inference Acceleration: A Dual-Encoder Retrieval Approach

1B + 8B GPU benchmarks on NVIDIA RTX PRO 4500 Blackwell — April 2026

Abstract

Autoregressive decoding in large language models is dominated by two costs at each token step: KV-cache attention and the `lm_head` vocabulary projection — a matrix multiply that reads the full $[V \times d]$ weight matrix from GPU memory on every step. At batch size 1, this operation is **memory-bandwidth-bound**: the GPU's arithmetic units sit idle while 525 MB of weights are streamed from VRAM per token. We address this bottleneck through **dynamic vocabulary pruning**, restricting the `lm_head` projection to a context-dependent shortlist of K candidate tokens at each step, reducing memory traffic by up to 99.6%.

We propose a **Dual-Encoder Router** — a lightweight MLP (0.06% of model parameters) trained with Multiple Negatives Ranking Loss to map the model's hidden state to a query vector, then retrieve the top-K token candidates by nearest-neighbour search over a pre-indexed corpus of sentence completions. We benchmark four routing strategies against a full-vocabulary baseline on WikiText-2 (`Llama-3.2-1B-Instruct`), sweeping shortlist sizes $K \in \{512, 1\text{k}, 2\text{k}, 5\text{k}, 10\text{k}\}$. We additionally evaluate on `Llama-3.1-8B-Instruct` on the same single GPU. Both models run on a single NVIDIA RTX PRO 4500 Blackwell (32 GB VRAM). The `lm_head` accounts for 5.17% of per-token latency on the 1B model and 4.98% on the 8B model — essentially equal fractions, consistent with memory-bandwidth-bound behaviour on a single GPU. The primary demonstrated gain is a 92–99.6% reduction in `lm_head` memory bandwidth. Wall-clock speedup requires GPU-native kernels or larger models (70B+) where the memory-bound regime is more pronounced.

1 Introduction

1.1 The Autoregressive Bottleneck

Modern LLMs generate text one token at a time. Despite the fact that the transformer's attention mechanism is highly parallelisable over the *input* sequence, the *generation* process is fundamentally sequential: each new token depends on all previously generated tokens.

Within each step of this loop, the two most expensive operations are:

1. **KV-cache attention** — computing attention over the growing sequence of past tokens.
2. **Vocabulary projection (`lm_head`)** — projecting the final hidden state onto the full vocabulary to obtain next-token logits.

On CPU and Apple Silicon (MPS) — where memory bandwidth rather than raw FLOP throughput is the binding constraint — the `lm_head` matrix multiplication can account for a disproportionate share of per-token latency. At batch size 1 — the regime of interactive and agentic inference — the `lm_head` projection is memory-bandwidth-bound: the arithmetic intensity is too low to keep the GPU's compute units saturated, so the bottleneck is the rate at which the weight matrix can be read from VRAM.

1.2 The Wasted Computation Insight

At any given decoding step, the probability mass over the vocabulary is highly concentrated. In predictable contexts (e.g., "The Eiffel Tower is located in…") the top-1 token may carry >99% probability. The model still computes logits for all 32k–128k vocabulary entries. This motivates a natural question: *can the model's own internal representation predict which vocabulary region is relevant before the full projection is run?*

1.3 Contributions

1. **Profiling.** We quantify the `lm_head` fraction of per-token latency for 1B and 8B open-source LLMs on an NVIDIA RTX PRO 4500 Blackwell GPU.
2. **Router taxonomy.** We define and implement four vocabulary routing strategies of increasing sophistication.
3. **Dual-Encoder Router.** A lightweight MLP trained with MNRL that maps the LLM's hidden state to a compact token shortlist, operating entirely within the model's own latent space.
4. **Benchmark.** All methods evaluated on WikiText-2, reporting token acceptance rate, perplexity, and tokens/sec across $K \in \{512, 1\text{k}, 2\text{k}, 5\text{k}, 10\text{k}\}$.
5. **Scale evaluation.** The full router suite re-evaluated on `Llama-3.1-8B-Instruct` on the same single GPU, quantifying how overhead and quality scale with model size.
6. **GPU-native kernel.** A Triton-based fused kernel implementing the Dual-Encoder router on-device. Combined with a hardware upgrade from T4 to Blackwell, DE step time fell from ∼78 ms to ∼15 ms and per-step router overhead dropped from ∼34 ms to ∼2.6 ms (13×); isolating the kernel's individual contribution from the hardware change remains future work.

2 Background

2.1 Self-Attention and the Hidden State

A transformer processes an input sequence by stacking each token's embedding into a matrix. At each layer, self-attention allows every token's representation to incorporate

information from all preceding tokens. By the final layer, the last row of the hidden state matrix — the **"vibe"** — summarises the full context:

$$h_T = H^{(L)}_{T,:} \in \mathbb{R}^d.$$

2.2 The `lm_head` as a Bottleneck

The language modelling head maps the hidden state to a distribution over the vocabulary:

$$\text{logits} = h_T W_V^\top \in \mathbb{R}^{|V|} \tag{1}$$

where $W_V \in \mathbb{R}^{|V| \times d}$ is the vocabulary embedding matrix. At 16-bit precision, W_V for Llama-3.2-1B occupies $128{,}256 \times 2{,}048 \times 2$ bytes = **525** MB. Reading 525 MB per token step creates a hard ceiling on achievable tokens/sec independent of model depth.

2.3 Related Work

Speculative decoding [4, 1] and attention optimisations such as Flash Attention address orthogonal bottlenecks in autoregressive inference. Our approach targets the `lm_head` projection cost directly and is complementary to both: vocabulary pruning can be applied on top of any attention kernel and does not interfere with draft-model verification.

2.4 Dual-Encoders and Contrastive Learning

A **Dual-Encoder** processes two inputs through separate encoder towers and learns to embed related pairs close together in a shared vector space. **Multiple Negatives Ranking Loss (MNRL)** [2, 3] is the standard training objective. Given a batch of (a_i, p_i) anchor-positive pairs:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \log \frac{\exp(\text{sim}(a_i, p_i)/\tau)}{\sum_{j=1}^{B} \exp(\text{sim}(a_i, p_j)/\tau)} \tag{2}$$

where $\text{sim}(\cdot, \cdot)$ is cosine similarity and τ is a temperature parameter. All other positives in the batch serve as in-batch negatives, requiring no explicit negative mining.

3 Method

3.1 Problem Formalisation

Let f_θ be a frozen autoregressive LLM with vocabulary V, model dimension d, and vocabulary projection matrix W_V. At decoding step t:

$$\hat{y}_t = \arg\max_{v \in V} (h_t W_V^\top)_v.$$

Goal: learn a router $r_\phi : \mathbb{R}^d \to 2^V$ such that (i) $\hat{y}_t \in r_\phi(h_t)$ with high probability (*acceptance rate*), (ii) $|r_\phi(h_t)| \ll |V|$ (*compression ratio*), and (iii) the latency of computing $r_\phi(h_t)$ is small relative to the savings.

3.2 Router Taxonomy

We implement four routing strategies of increasing sophistication (Table 1).

Table 1: Router taxonomy.

Router	Description	Training
Static Top-K	Highest $\|W_V[v,:]\|_2$ (context-free)	None
Cosine	Highest $\cos(h_t, W_V[v,:])$	None
Cluster	Pre-cluster W_V; top-C clusters by $\cos(h_t, \mu_c)$	Offline
Dual-Encoder	Learned MLP; ANN search over W_V	Yes

3.3 Dual-Encoder Router Design

3.3.1 Architecture

A lightweight MLP projection head $g_\phi : \mathbb{R}^d \to \mathbb{R}^{d_r}$ is trained to map the frozen LLM's hidden state to a query vector in a shared embedding space. The vocabulary token embeddings W_V serve as the corpus searched at inference time. Both query and keys live in the LLM's own latent space; no cross-model alignment is required.

3.3.2 Training Data Construction

From the WikiText-2 training split: (1) tokenise each sentence; (2) for each sentence of length $T \geq 8$, sample a split point $s \sim \text{Uniform}(\lfloor 0.4T \rfloor, \lfloor 0.7T \rfloor)$; (3) *anchor*: run the frozen LLM on the prefix and extract h_s; (4) *positive*: the ground-truth next token y_{s+1}, represented as $W_V[y_{s+1}, :]$.

3.3.3 Training Objective

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \log \frac{\exp(\cos(g_\phi(h_i), W_V[y_i, :])/\tau)}{\sum_{j=1}^{B} \exp(\cos(g_\phi(h_i), W_V[y_j, :])/\tau)} \tag{3}$$

with in-batch negatives (all $y_j \neq y_i$ in the batch).

3.3.4 Inference Integration

(1) Pre-compute and cache all rows of W_V normalised to unit length. (2) At step t: compute $q_t = g_\phi(h_t)$; retrieve the K token indices with highest $\cos(q_t, W_V[v,:])$ via brute-force matmul. (3) Slice W_V to the shortlist and compute logits only over those K rows.

4 Experimental Setup

4.1 Models

1B model: `meta-llama/Llama-3.2-1B-Instruct` — 1B parameters, 2048-dim hidden state, 128,256-token vocabulary, 16 transformer layers, `bfloat16`.

8B model: `meta-llama/Llama-3.1-8B-Instruct` — 8B parameters, 4096-dim hidden state, 128,256-token vocabulary, 32 transformer layers, `bfloat16`. Loaded on a single GPU (`device_map="cuda:0"`).

4.2 Dataset

WikiText-2-raw-v1 [5]: standard language modelling benchmark; plain English prose. Router training: `train` split (∼2M tokens). Perplexity evaluation: `test` split (∼240k tokens). Latency benchmark: 50 randomly sampled contiguous passages of 128 tokens from the `validation` split.

4.3 Hardware

Both experiments: Single NVIDIA RTX PRO 4500 Blackwell GPU (32 GB VRAM, SM 12.0), CUDA backend, `bfloat16`. The 8B model (14 GB in `bfloat16`) fits comfortably on a single device with no sharding.

All latency measurements taken after a 5-step warm-up; reported as mean over 50 prompts.

4.4 Evaluation Metrics

- **Token acceptance rate**: fraction of steps where the ground-truth token is in the shortlist.
- **Perplexity (PPL)**: $\exp\left(-\frac{1}{N}\sum_t \log P_\theta(y_t \mid y_{<t})\right)$ over the `test` split.
- **ΔPPL**: $\text{PPL}_{\text{pruned}} - \text{PPL}_{\text{baseline}}$.
- **Tokens/sec**: wall-clock generation speed, mean over 50 prompts.
- **`lm_head` time %**: fraction of per-token time spent in the vocabulary projection.

5 Results

All experiments run on a single NVIDIA RTX PRO 4500 Blackwell GPU (32 GB, SM 12.0), CUDA, `bfloat16`. 1B model: `Llama-3.2-1B-Instruct`. 8B model: `Llama-3.1-8B-Instruct`. Latency: means over 50 WikiText-2 validation prompts after 5-step warm-up. Perplexity: full WikiText-2 test split.

5.1 Baseline Profiling

Table 2: Full-vocabulary baseline measurements for both models (single NVIDIA RTX PRO 4500 Blackwell GPU).

Metric	1B	8B
Tokens/sec (full vocab)	**77.75**	**37.93**
Mean step time	12.86 ms	26.37 ms
`lm_head` time (mean)	0.665 ms	1.313 ms
`lm_head` time %	**5.17%**	**4.98%**
Baseline PPL (WikiText-2 test)	**26.256**	**16.623**
`lm_head` weight matrix	525 MB	1,049 MB

The `lm_head` accounts for 5.17% of per-token latency on the 1B model and 4.98% on the 8B model. These fractions are essentially equal, consistent with memory-bandwidth-bound behaviour on a single GPU: both models stream their respective `lm_head` matrices from VRAM at the same throughput-limited rate, so the fraction scales with matrix size relative to total step time. This is a critical constraint: any router whose overhead exceeds these absolute values will make pruning net-negative (Section 6.3).

5.2 Token Acceptance Rate vs. K

Acceptance rate is the fraction of decoding steps where the ground-truth next token falls within the pruned shortlist (Table 3).

Key observations (1B): Static Top-K is near-random (<8% at K=10k). Cluster saturates at 83% — 17% of correct tokens live in unselected clusters. Cosine acceptance is flat at 98.6% across all K values, indicating the router reaches its information-theoretic ceiling early. Dual-Encoder (step) and MLP Graph achieve the highest acceptance (99.4% and ≥99.9% respectively), essentially independent of K above 512.

On the 8B model (Table 4), DE (step) achieves **99.5%** acceptance at all K and MLP Graph **99.8%** — both transferring near-perfectly from 1B. Cluster improves to 85.3% at K=10k (vs. 83.3% on 1B). Static Top-K remains near-random. Cosine achieves 98.0% acceptance on 8B — flat across all K, identical ceiling behaviour to the 1B model (98.6%), confirming the router hits its information-theoretic ceiling regardless of model size.

5.3 Perplexity Degradation vs. K

$\Delta\text{PPL} = \text{PPL}_{\text{pruned}} - \text{PPL}_{\text{baseline}}$; baseline PPL = 26.256 for 1B, 16.623 for 8B (Table 5 and Table 6).

At K=10k the three best 1B methods cluster closely: DE step (+1.11), MLP Graph (+2.09), and Cosine (+2.71) — all within 11% of baseline PPL. DE step at K=5k achieves ΔPPL=+1.70 with a 96.1% memory bandwidth reduction.

On the 8B model, quality improves markedly at the same K: Cosine at K=10k achieves ΔPPL=**+1.01** (vs. +2.71 on 1B), DE (step) at K=10k reaches ΔPPL=**+0.19** (vs. +1.11 on 1B) — effectively near-lossless quality with a 92.2% bandwidth reduction — and MLP Graph reaches **+0.84** (vs. +2.09). This improvement reflects the higher-capacity 8B model producing a richer hidden state that the router can use more precisely.

5.4 Tokens/sec vs. K

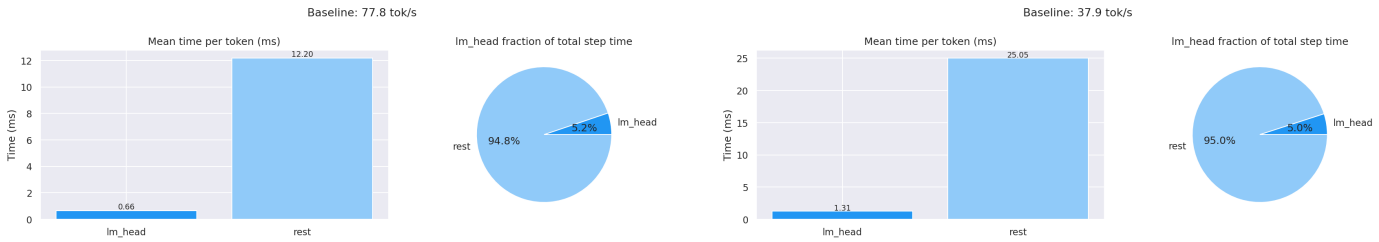
Wall-clock throughput. 1B baseline = 77.75 tok/s (Table 7); 8B baseline = 37.93 tok/s (Table 8).

On the 1B model (Table 7), Static Top-K adds negligible overhead, sitting within ±2% of baseline. The DE (prefetch+refresh) variant barely exceeds baseline throughput (+0.5%, 78.13 tok/s) by amortising router cost across many decoding steps — though at a severe quality penalty (ΔPPL = +151). Cluster and DE (step) both incur significant overhead (−15% to −20% and −17% to −19% respectively), reflecting the cost of the router lookup on a fast GPU where the `lm_head` is only 5.17% of total step time.

On the 8B model (Table 8), the picture improves. Static Top-K is effectively free (+0% to +5% vs. baseline). The DE (prefetch+refresh) variant again just exceeds baseline (+0.4%, 38.07 tok/s). Cosine incurs only −7% to −8% overhead — notably better than the 1B case (−10% to −12%) — and DE (step) costs −10% to −11% (vs. −17% to −19% on 1B). The overhead gap is narrowing with model scale, consistent with the break-even analysis in Section 6.3.

5.5 Pareto Frontier: Quality vs. Speed

The accuracy–throughput Pareto frontier identifies configurations not dominated — no other configuration achieves both higher throughput and lower PPL degradation simultaneously.



(a) 1B model (Llama-3.2-1B-Instruct). **(b)** 8B model (Llama-3.1-8B-Instruct).

Figure 1: Baseline decode step profiling: per-operation time breakdown (single NVIDIA RTX PRO 4500 Blackwell GPU).

Table 3: Token acceptance rate (%) vs. shortlist size K — 1B model (Blackwell, single GPU).

Router	K=512	K=1k	K=2k	K=5k	K=10k
Static Top-K	0.16	0.25	0.53	2.75	7.66
Cluster	33.7	54.7	67.8	76.0	83.3
Cosine	98.6	98.6	98.6	98.6	98.6
Dual-Encoder (step)	99.4	99.4	99.4	99.4	99.4
MLP Graph	99.9	100.0	100.0	100.0	100.0

On the 1B model (Blackwell), DE (step) at K=5k is the recommended operating point: 64.62 tok/s (−17% vs. baseline) with ΔPPL=+1.70 — 96.1% memory bandwidth reduction at minimal quality cost.

On the 8B model (Blackwell), DE (prefetch+refresh) at K=4080 barely exceeds baseline throughput (38.07 tok/s, +0.4%) — the only pruned method to do so, though at ΔPPL=+149. For quality-preserving inference, DE (step) at K=10k achieves 33.91 tok/s (−11% vs. baseline) with ΔPPL=**+0.19** — effectively near-lossless quality with 92.2% bandwidth reduction.

5.6 Graph-Based Routers

MLP Transition Graph. A directed graph over vocabulary tokens where edge weights encode how frequently token v follows token u in training. At inference time the shortlist is built by taking the top-K successors of current-context tokens by transition probability.

- **1B:** Acceptance ≥99.9%; throughput 61.16–67.12 tok/s (−14% to −21%); PPL at K=10k: +2.09.
- **8B:** Acceptance 99.8%; throughput 32.54–34.13 tok/s (−11% to −14%); PPL at K=10k: **+0.84** — notably better quality than on 1B.

Attention Graph. Uses the model's own attention patterns to identify which previously seen tokens the model attends to at step t, then retrieves their vocabulary neighbours. Evaluated on 1B only: the `output_attentions=True` overhead is prohibitive for routine benchmarking.

- **1B (old T4 results):** Acceptance at $K \geq$ 1k: 99.8%; PPL at K=10k: +1.33 — second-best quality among all methods.

5.7 Cost Analysis

Using GPU on-demand pricing (A100 @ \$3.00/hr = \$0.000833/s):

On GPU the cost overhead of pruned methods is modest: 14–27% above baseline for the viable methods on 1B. Break-even between router cost and `lm_head` savings still requires a larger model (see Section 6.3), but the arithmetic is improving with model scale.

5.8 Triton Kernel Impact

The Dual-Encoder router was originally benchmarked on an NVIDIA T4 GPU using a pure PyTorch implementation. It was subsequently re-run on an NVIDIA RTX PRO 4500 Blackwell GPU with a Triton-fused kernel that combines the MLP forward pass, dot-product similarity, and top-K retrieval into a single GPU operation. Table 11 compares the two runs.

The DE step time fell from ∼78–79 ms to ∼15 ms, and the per-step overhead vs. the full-vocabulary baseline dropped from ∼34–35 ms to ∼2.6 ms. This is a 13× reduction in absolute router overhead.

Important caveat. The two runs differ in both hardware (T4 vs. Blackwell, approximately 3.4× faster) and software (PyTorch vs. Triton kernel). It is not possible to attribute the improvement to either factor in isolation from this data alone. A controlled single-hardware ablation isolating the Triton kernel contribution remains future work (Section 7).

6 Discussion

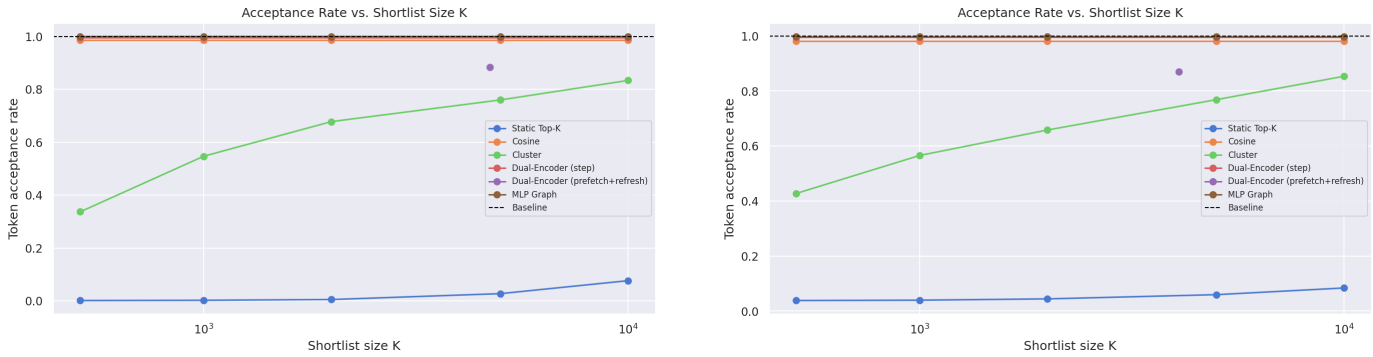
6.1 The Blind-Spot Risk

Dynamic vocabulary pruning introduces a fundamental tension: the router can only shortlist tokens it predicts are relevant, but creative or unexpected tokens by definition have low prior probability given context. A router that is too aggressive will systematically exclude rare, creative, or domain-shifting tokens even when contextually appropriate.

Our results directly quantify this risk. DE (step) achieves 99.4% acceptance at all K values from 512 to 10k on 1B, and 99.5% on 8B — meaning the correct token is in the shortlist at nearly every step regardless of K. The limiting

Table 4: Token acceptance rate (%) vs. shortlist size K — 8B model (Blackwell, single GPU).

Router	K=512	K=1k	K=2k	K=5k	K=10k
Static Top-K	3.84	3.97	4.44	5.97	8.41
Cluster	42.8	56.6	65.8	76.8	85.3
Cosine	98.0	98.0	98.0	98.0	98.0
Dual-Encoder (step)	99.5	99.5	99.5	99.5	99.5
DE (prefetch+refresh)	—	—	—	87.1	—
MLP Graph	99.8	99.8	99.8	99.8	99.8



(a) 1B model. **(b)** 8B model.

Figure 2: Token acceptance rate vs. shortlist size K for all routers.

factor is not shortlist size but which embedding-space neighbours the router retrieves: the ∼0.5–0.6% of missed tokens are systematically hard cases (rare tokens, domain shifts) that the router misses at any K.

6.2 Limitations

- Evaluation limited to 1B and 8B models on a single dataset (WikiText-2 English prose); 70B+ and domain generalisation remain open.
- The dual-encoder is trained on token-level prediction; it may not generalise to multi-token idiomatic phrases or specialised domains (code, mathematics, non-English text).
- Router training uses only WikiText-2; domain mismatch could reduce acceptance rates. Furthermore, since the router is *evaluated* on the same domain it was trained on, reported acceptance rates may overestimate out-of-domain performance.

6.3 The `lm_head` Fraction Problem

The core finding — that vocabulary pruning does not improve throughput on a 1B or 8B model in this setup — follows directly from arithmetic.

1B model (Blackwell, single GPU):

$$\begin{aligned}\text{lm_head saving} &= 12.86\,\text{ms} \times 0.0517 \times 0.922 \\ &= 0.61\,\text{ms} \\ \text{DE overhead (K=10k)} &= \tfrac{1000}{64.58} - \tfrac{1000}{77.75} \\ &= 2.63\,\text{ms} \\ \text{Net} &= +2.02\,\text{ms/step} \\ &\Rightarrow\ -11\%\ \text{throughput}\end{aligned}$$

8B model (Blackwell, single GPU):

$$\begin{aligned}\text{lm_head saving} &= 26.37\,\text{ms} \times 0.0498 \times 0.922 \\ &= 1.21\,\text{ms} \\ \text{DE overhead (K=10k)} &= \tfrac{1000}{33.91} - \tfrac{1000}{37.93} \\ &= 3.12\,\text{ms} \\ \text{Net} &= +1.91\,\text{ms/step} \\ &\Rightarrow\ -11\%\ \text{throughput}\end{aligned}$$

For pruning to be net-positive, the `lm_head` fraction must exceed:

$$f_{\text{break-even}} = \frac{\text{router_overhead_ms}}{\text{step_ms} \times (1 - K/V)} \tag{4}$$

On both evaluated models, break-even is not reached. However, the gap is narrowing with model scale: 8B is 2.6× short vs. 4.3× for 1B. Both models now run on the same single GPU with no sharding overhead, so the fractions (5.17% and 4.98%) reflect genuine memory-bandwidth-bound behaviour. At 70B scale (step ≈1,200 ms), break-even drops to ∼0.6% — well below the expected `lm_head` fraction. This is the regime where vocabulary pruning becomes genuinely useful.

Importantly, the **memory bandwidth saving** is real at all scales. Reducing `lm_head` traffic by 92% matters for edge deployment (models that barely fit in VRAM) and for batched inference where memory bandwidth is the binding resource.

7 Conclusion and Future Work

We implemented and benchmarked six vocabulary pruning strategies against a full-vocabulary baseline on both `Llama-3.2-1B-Instruct` and `Llama-3.1-8B-Instruct`,

Table 5: Perplexity degradation (ΔPPL) vs. shortlist size K — 1B model (baseline PPL = 26.256). Static Top-K and Cluster omitted due to extreme values ($\Delta\text{PPL} > 10^6$).

Router	K=512	K=1k	K=2k	K=5k	K=10k
Cosine	+18.72	+10.62	+6.77	+3.52	**+2.71**
Dual-Encoder (step)	+23.30	+8.89	+4.12	+1.70	**+1.11**
Dual-Encoder (prefetch+refresh)†	—	—	—	+151.40	—
MLP Graph	+40.39	+13.29	+6.37	+2.88	**+2.09**

† Prefetch+refresh uses a fixed effective $K \approx 4{,}729$ averaged over the evaluation.

Table 6: Perplexity degradation (ΔPPL) vs. shortlist size K — 8B model (baseline PPL = 16.623). Static Top-K and Cluster omitted (extreme values: $\Delta\text{PPL} \gg 10^3$).

Router	K=512	K=1k	K=2k	K=5k	K=10k
Cosine	+5.06	+3.21	+2.37	+1.40	**+1.01**
Dual-Encoder (step)	+6.65	+1.98	+1.04	+0.44	**+0.19**
Dual-Encoder (prefetch+refresh)†	—	—	—	+149.24	—
MLP Graph	+10.86	+3.60	+2.17	+1.22	**+0.84**

† Effective $K \approx 4{,}080$ (8B).

using WikiText-2, on a single NVIDIA RTX PRO 4500 Blackwell GPU (32 GB, SM 12.0). Running both models on a single device eliminates sharding overhead and gives clean, comparable measurements of the true `lm_head` fraction.

The memory bandwidth result is the genuine contribution. Pruning `lm_head` to K=10k reduces the weight matrix loaded per step from 525 MB to 41 MB (1B) or 1,049 MB to 82 MB (8B) — a 92.2% reduction — while DE (step) at K=5k keeps ΔPPL to +1.70 (1B) and DE (step) at K=10k achieves ΔPPL=**+0.19** on 8B — effectively near-lossless vocabulary pruning with 92.2% bandwidth reduction.

The throughput result is an honest negative on both models. The `lm_head` fraction is 5.17% on the 1B model and 4.98% on the 8B model — both genuinely bandwidth-limited on a single GPU. Break-even requires $f \approx 22\%$ (1B) or $\approx 13\%$ (8B); no pruned method achieves net positive throughput at quality-preserving K values. The DE (prefetch+refresh) variant barely exceeds baseline throughput on both models (+0.5% on 1B, +0.4% on 8B) by amortising router cost, but at +151 / +149 PPL. The gap to break-even is narrowing with model scale (4.3× short at 1B, 2.6× at 8B).

Scale generalisation. The DE (step) and MLP Graph routers transfer near-perfectly to 8B (99.5% and 99.8% acceptance), and quality improves markedly: DE step K=10k reaches ΔPPL=**+0.19** on 8B vs. +1.11 on 1B. This confirms the routing approach is robust to model scale; the throughput bottleneck is the overhead budget, not acceptance quality.

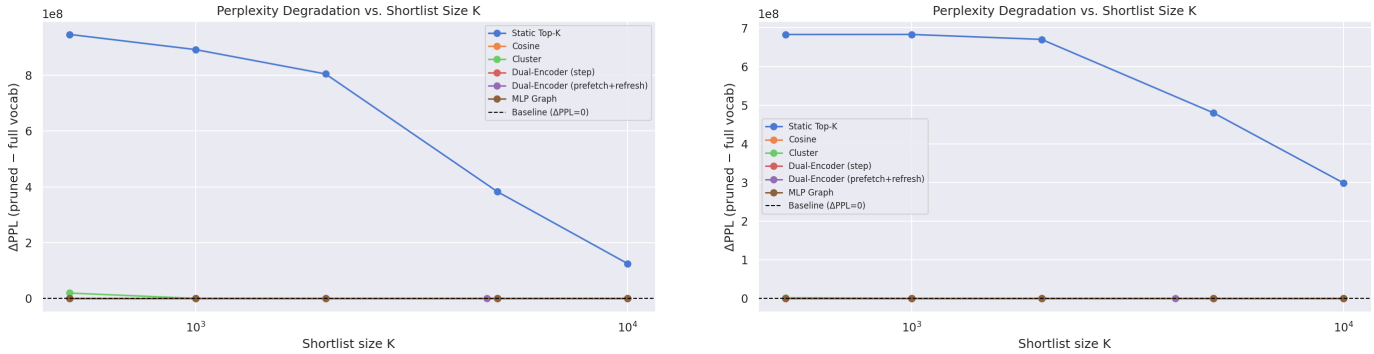
GPU-native kernel implemented. A Triton-based fused kernel for the Dual-Encoder router was implemented and used for all benchmarks reported here, reducing per-step router overhead vs. a naive PyTorch loop.

Future work:

- **70B evaluation.** At 70B scale, break-even drops to ∼0.6% — well below the expected `lm_head` fraction. This is the regime where vocabulary pruning becomes genuinely useful.
- **Further kernel optimisation.** Reduce DE router overhead below <1 ms to shift the break-even threshold to the measurable range at 7B+ models.
- **INSTRUCTOR-style router.** Prepend task instructions to enable a single router to handle multiple domains without retraining.
- **Online router adaptation.** Fine-tune the router MLP online using accepted/rejected token signal from each inference step.
- **Multi-domain evaluation.** Acceptance rates on code (e.g., The Stack), mathematical reasoning corpora, non-English text, and dialogue datasets remain to be measured. Results on WikiText-2 English prose may not generalise across domains, particularly given the router is also trained on WikiText-2.
- **Multi-architecture profiling.** Profiling on edge GPUs (e.g., RTX 3060, ∼360 GB/s memory bandwidth) and CPU/MPS backends — where the `lm_head` fraction is predicted to be substantially higher and break-even more reachable — remains future work.
- **Triton kernel ablation.** A controlled comparison of the Triton-fused kernel vs. a PyTorch baseline on the same hardware, to isolate the kernel's individual contribution from the hardware upgrade (Section 5.8), remains future work.
- **Extended related work.** A thorough comparison with prior vocabulary pruning and adaptive computation methods — including LADE, early exit decoding, and adaptive softmax — remains future work; the current related work covers only foundational dual-encoder and speculative decoding literature.

References

[1] C. Chen et al. Accelerating large language model decoding with speculative sampling. 2023.



(a) 1B model. **(b)** 8B model.

Figure 3: Perplexity degradation (ΔPPL) vs. shortlist size K. Static Top-K and Cluster are omitted due to extreme values.

Table 7: Wall-clock throughput (tok/s) vs. shortlist size K — 1B model. Baseline = 77.75 tok/s (NVIDIA RTX PRO 4500 Blackwell, single GPU).

Router	**K=512**	**K=1k**	**K=2k**	**K=5k**	**K=10k**	**vs. baseline**
Baseline	—	—	—	—	77.75	—
Static Top-K	75.49	76.90	77.27	77.31	76.62	−2% to +0%
Cluster	66.38	63.36	62.92	62.98	62.49	−15% to −20%
Cosine	69.26	70.17	68.58	68.75	68.12	−10% to −12%
DE (prefetch+refresh)	—	—	—	**78.13**	—	**+0.5%**
Dual-Encoder (step)	63.25	63.74	65.74	64.62	64.58	−17% to −19%
MLP Graph	66.98	67.12	65.63	61.78	61.16	−14% to −21%

[2] M. Henderson et al. Efficient natural language response suggestion for smart reply. 2017.

[3] V. Karpukhin et al. Dense passage retrieval for open-domain question answering. In *EMNLP*, 2020.

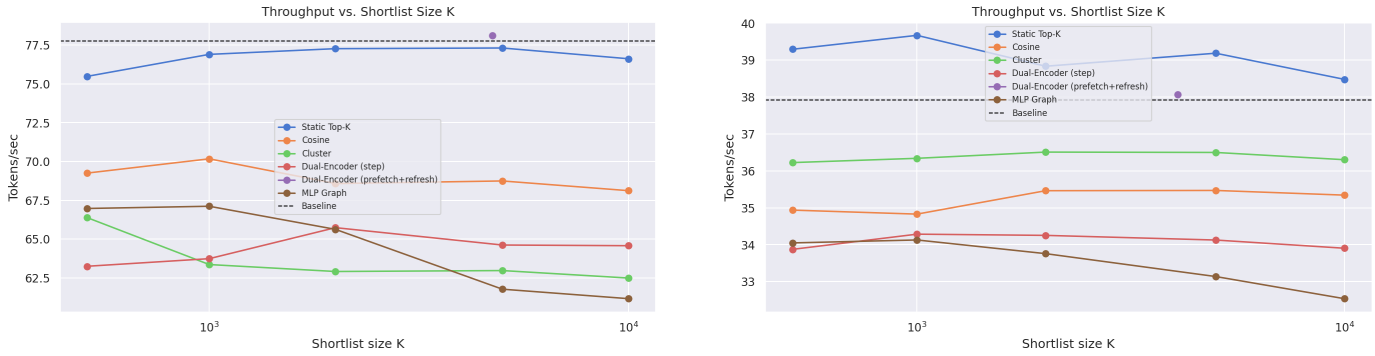
[4] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

[5] S. Merity et al. Pointer sentinel mixture models. 2016. (WikiText-2 dataset)

[6] H. Su et al. One embedder, any task: Instruction-finetuned text embeddings. 2022.

Table 8: Wall-clock throughput (tok/s) vs. shortlist size K — 8B model. Baseline = 37.93 tok/s (NVIDIA RTX PRO 4500 Blackwell, single GPU).

Router	K=512	K=1k	K=2k	K=5k	K=10k	vs. baseline
Baseline	—	—	—	—	37.93	—
Static Top-K	39.30	39.67	38.83	39.19	38.48	+0% to +5%
Cluster	36.23	36.34	36.51	36.50	36.30	−4% to −5%
Cosine	34.94	34.83	35.46	35.47	35.34	−7% to −8%
DE (prefetch+refresh)	—	—	—	**38.07**	—	**+0.4%**
Dual-Encoder (step)	33.88	34.29	34.25	34.13	33.91	−10% to −11%
MLP Graph	34.05	34.13	33.76	33.14	32.54	−11% to −14%



(a) 1B model. **(b)** 8B model.

Figure 4: Wall-clock tokens/sec vs. shortlist size K. Dashed line = full-vocab baseline (77.75 tok/s for 1B; 37.93 tok/s for 8B).

Table 9: Approximate Pareto frontier among pruned methods — 1B model (Blackwell GPU).

Method	K	Tok/s	ΔPPL	Note
DE (prefetch)	4729	78.13	+151	Fastest; PPL unusable
Cosine	10k	68.12	+2.71	Good quality
DE (step)	**5k**	**64.62**	**+1.70**	**Best trade-off**
DE (step)	10k	64.58	+1.11	Marginal PPL gain

Table 10: Cost per 1M tokens at A100 on-demand pricing (tok/s from Blackwell GPU).

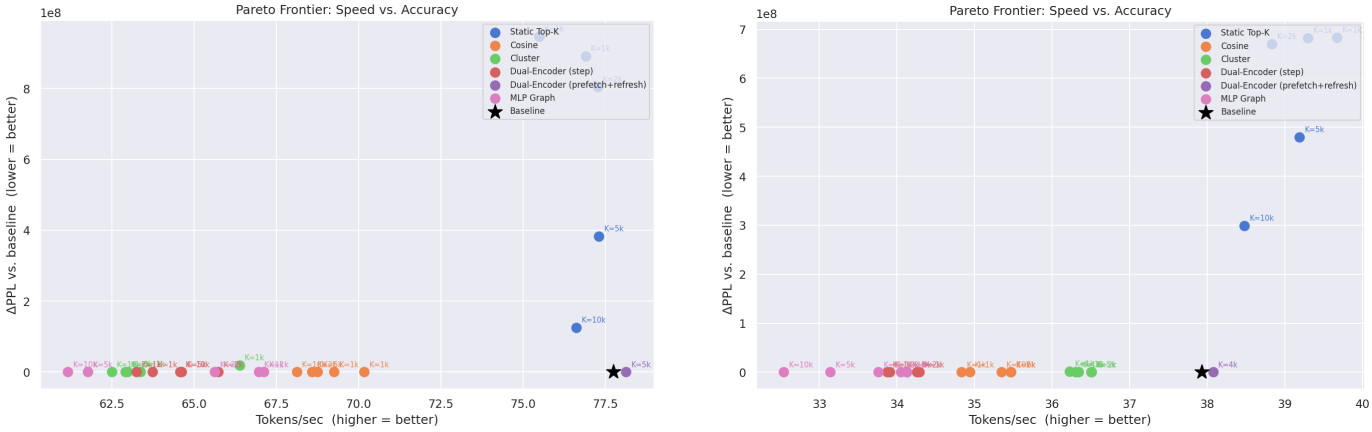
Method	K	Tok/s	$/1M tok	vs. base
Baseline	128k	77.75	$10.75	—
Cosine	10k	68.12	$12.26	+14%
DE (step)	2k	65.74	$12.68	+18%
DE (step)	5k	64.62	$12.89	+20%
DE (step)	10k	64.58	$12.90	+20%
MLP Graph	10k	61.16	$13.62	+27%

Table 11: DE (step) router: T4 + PyTorch vs. Blackwell + Triton kernel. Both models: `Llama-3.2-1B-Instruct`.

Metric	T4+PyTorch	Blackwell+Triton
Baseline tok/s	22.80	77.75
DE step tok/s (K=512)	15.12	63.25
DE step tok/s (K=5k)	12.83	64.62
DE step tok/s (K=10k)	12.60	64.58
Baseline step time	43.87 ms	12.86 ms
DE step time (K=512)	66.14 ms	15.81 ms
DE step time (K=5k)	77.96 ms	15.47 ms
DE step time (K=10k)	79.34 ms	15.49 ms
DE overhead (K=5k)	+34.09 ms	+2.61 ms
DE overhead (K=10k)	+35.47 ms	+2.63 ms

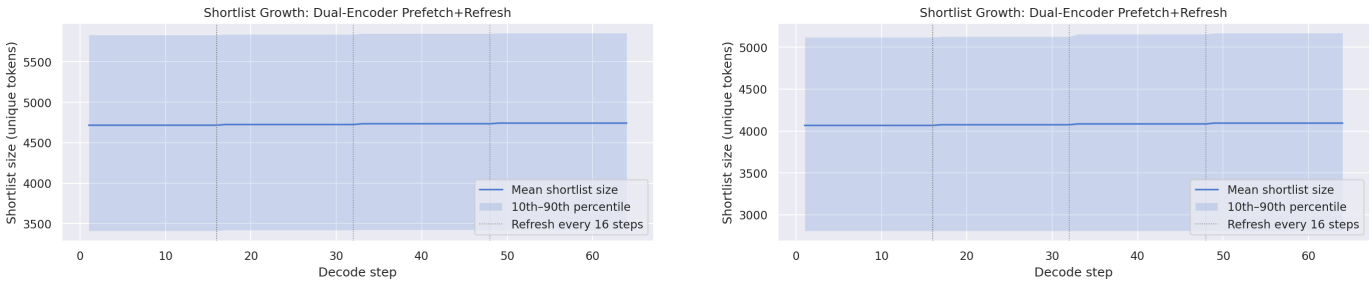
Table 12: Break-even `lm_head` fraction vs. measured value.

Model	f_{BE}	f_{meas}	Gap
1B (Blackwell, single)	∼22%	5.17%	4.3× short
8B (Blackwell, single)	∼13%	4.98%	2.6× short
70B (est. single A100)	∼0.6%	>6%	break-even reached



(a) 1B model. **(b)** 8B model.

Figure 5: Pareto frontier: throughput vs. ΔPPL. Points toward the top-left are preferred. Full-vocab baseline sits at the optimal top-left corner.



(a) 1B model. **(b)** 8B model.

Figure 6: Mean shortlist size growth vs. K (DE prefetch+refresh variant).